Traineeship Application

Sprint Report

<Georgios Marinos Papadopoulos , AM : 4460>

<Dimitrios Georgitsis , AM : 4334>

<Nikolaos Chatzoulis , AM : 4524>

VERSIONS HISTORY

Date	Version	Description	Author
25/5/2025	Final	Final Report	4524 , 4460, 4334

1 Introduction

This document provides information concerning the final sprint of the project.

1.1 Purpose

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes out Scrum team and specifies the this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Apostolos Zarras
Scrum Master	Apostolos Zarras
Development Team	Chatzoulis Nikolaos , Dimitrios Georgitsis , Georgios Marinos Papadopoulos

2.2 Sprints

Sprint No	Begin Date	End Date	Number of weeks	User stories
1	**15/4**	**22/4**	**1**	**1,2,3**
2	**23/4**	**7/5**	**2**	**4,5,7,8,9,10,11**
3	**8/5**	**16/5**	**1**	**13,14,16,17,18,19**
4	**17/5**	**25/5**	**1**	**6,12,15,20,21**

3 Use Cases

3.1 <Use Case 1>

Use case ID	1
Actors	User
Pre conditions	-
Main flow of events	1. The use case starts when the user press the signup button. 2. The system shows up the signup form 3. The user fills in the requested data and presses the signup button 4. The system saves the user
Alternative flow 1	If the user already exists then the system informs the user.
Post conditions	The user's database has one more entry.

3.2 <Use Case 2>

Use case ID	2
Actors	User
Pre conditions	The user is already signed up.
Main flow of events	1. The use case starts when the user press the login button. 2. The system shows up the login form 3. The user fills in the requested data and presses the login button 4. The system login the user to the app

Alternative flow 1	If the user fills in wrong credentials the system informs the user.
Post conditions	-

3.3 <Use Case 3>

Use case ID	3
Actors	User
Pre conditions	The user is already signed up.
Main flow of events	The use case starts when the user presses the logout button. The system logs out the user
Alternative flow 1	-
Post conditions	-

3.4 <Use Case 4>

Use case ID	4
Actors	Student
Pre conditions	The user is already signed up.
Main flow of events	1. The use case starts when the student press the profile button. 2. The system shows up the profile form 3. The student fills in the requested data and presses the save button 4. The system saves the student profile

Alternative flow 1	If the student's profile already exists then the system doesn't save the new profile info.
Post conditions	The student profile is created.

3.5 <Use Case 5>

Use case ID	5
Actors	Student
Pre conditions	-
Main flow of events	1. The use case starts when the student presses the apply for traineeship button. 2. The system informs the student object.
Alternative flow 1	-
Post conditions	The student is available for assigning a traineeship position.

3.6 <Use Case 6>

Use case ID	6
Actors	Student
Pre conditions	The student must have a traineeship position assigned.
Main flow of events	1. The use case starts when the student presses the Fill Logbook button. 2. The system shows up the form 3. The student fills in the requested data and presses the save button

	4. The system saves the logbook
Alternative flow 1	-
Post conditions	The logbook of the assigned traineeship position is filled.

3.7 <Use Case 7>

Use case ID	7
Actors	Company
Pre conditions	Signed in as company
Main flow of events	1. The use case starts when the company presses the profile button. 2. The system shows up the profile form 3. The company fills in the requested data and presses the save button 4. The system saves the company's profile
Alternative flow 1	If the profile already exists then the system doesn't save the new profile info.
Post conditions	The company profile is created.

3.8 <Use Case 8>

Use case ID	8
Actors	Company
Pre conditions	Signed in as company
Main flow of events	1. The use case starts when the user presses the available traineeships button.

	2. The system shows up the list of the available traineeships.
Alternative flow 1	-
Post conditions	-

3.9 <Use Case 9>

Use case ID	9
Actors	Company
Pre conditions	Signed in as company
Main flow of events	1. The use case starts when the user presses the assigned traineeships button. 2. The system shows up the list of the assigned traineeships.
Alternative flow 1	-
Post conditions	-

3.10 <Use Case 10>

Use case ID	10
Actors	Company
Pre conditions	Signed in as company
Main flow of events	1. The use case starts when the user presses the announce traineeship button. 2. The system shows up the traineeship form 3. The user fills in the requested data and presses the announce button

	4. The system saves the new traineeship position
Alternative flow 1	-
Post conditions	The new traineeship position is added to the list of the available positions.

3.11 <Use Case 11>

Use case ID	11
Actors	Company
Pre conditions	Signed in as company
Main flow of events	1. The use case starts when the user presses the delete button of a traineeship position. 2. The system shows up a confirmation dialog. 3. The company presses the delete button on confirmation dialog. 4. The system deletes the traineeship position.
Alternative flow 1	If the company presses the cancel button then the system do nothing.
Post conditions	The list of traineeship positions decreased by one.

3.12 <Use Case 12>

Use case ID	12
Actors	Company
Pre conditions	Signed in as company
Main flow of events	1. The use case starts when the company presses the evaluation button of

	an assigned traineeship position. 2. The system shows up the evaluation form 3. The company fills in the requested data and presses the evaluate button 4. The system saves the company's evaluation
Alternative flow 1	-
Post conditions	The evaluation is created.

3.13 <Use Case 13>

Use case ID	13
Actors	Professor
Pre conditions	Signed in as professor
Main flow of events	1. The use case starts when the professor presses the profile button. 2. The system shows up the profile form 3. The professor fills in the requested data and presses the save button 4. The system saves the professor's profile
Alternative flow 1	If the profile already exists then the system doesn't save the new profile info.
Post conditions	The professor's profile is created.

3.14 <Use Case 14>

Use case ID	14
Actors	Professor
Pre	Signed in as professor

conditions	
Main flow of events	1. The use case starts when the professor presses the supervised traineeships button. 2. The system shows up the supervised traineeship positions for the professor.
Alternative flow 1	-
Post conditions	-

3.15 <Use Case 15>

Use case ID	15
Actors	Professor
Pre conditions	Signed in as professor
Main flow of events	1. The use case starts when the professor presses the evaluation button of a supervised traineeship position. 2. The system shows up the evaluation form 3. The professor fills in the requested data and presses the evaluate button 4. The system saves the evaluation
Alternative flow 1	-
Post conditions	The evaluation is created.

3.16 <Use Case 16>

Use case ID	16

Actors	Committee member
Pre conditions	Signed in as committee member
Main flow of events	1. The use case starts when the member presses the applied users button. 2. The system shows up the users that have applied for a traineeship position.
Alternative flow 1	-
Post conditions	-

3.17 <Use Case 17>

Use case ID	17
Actors	Committee member
Pre conditions	Signed in as committee member
Main flow of events	1. The use case starts when the member presses one of the three choices for position searching. 2. The system shows the list of available positions.
Alternative flow 1	-
Post conditions	-

3.18 <Use Case 18>

Use case ID	18
Actors	Committee member

Pre conditions	Signed in as committee member
Main flow of events	1. The use case starts when the user presses the assign button of a position. 2. The system assign the position to the student.
Alternative flow 1	-
Post conditions	The position is assigned to the student.

3.19 <Use Case 19>

Use case ID	19
Actors	Committee member
Pre conditions	Signed in as committee member
Main flow of events	1. The use case starts when the user presses the one of the choices for assign supervisor. 2. The system assign the supervisor with the chosen strategy.
Alternative flow 1	-
Post conditions	The professor is assigned to a traineeship position.

3.20 <Use Case 20>

Use case ID	20
Actors	Committee member

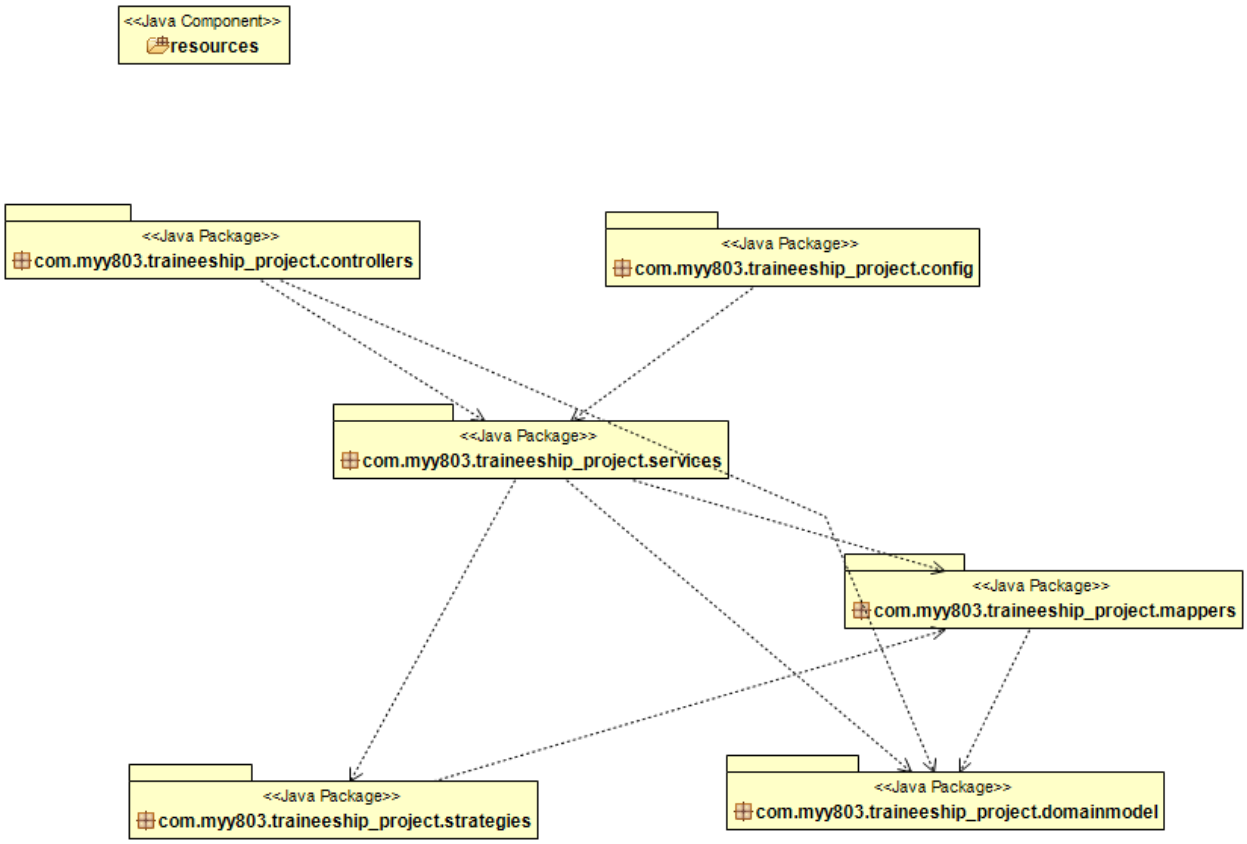
Pre conditions	Signed in as committee member
Main flow of events	1. The use case starts when the professor presses the in-progress traineeships button. 2. The system shows the in-progress traineeship positions.
Alternative flow 1	-
Post conditions	-

3.21 <Use Case 21>

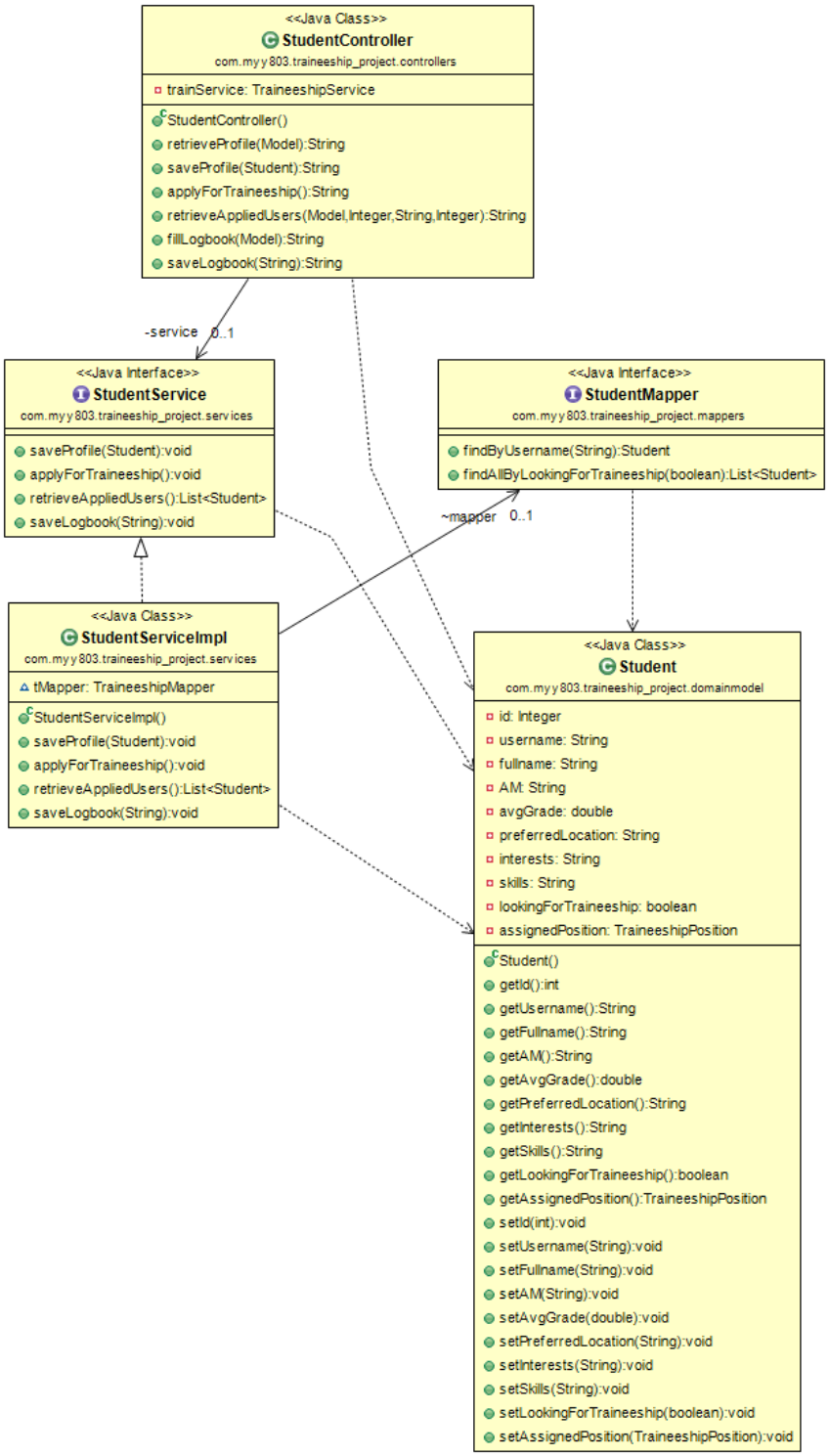
Use case ID	21
Actors	Committee member
Pre conditions	Signed in as committee member
Main flow of events	1. The use case starts when the member presses the show evaluation of an in-progress traineeship position. 2. The system shows up the evaluation. 3. The member chooses the pass or fail grade and presses the submit button. 4. The system saves the grade.
Alternative flow 1	-
Post conditions	The member assign the position grade.

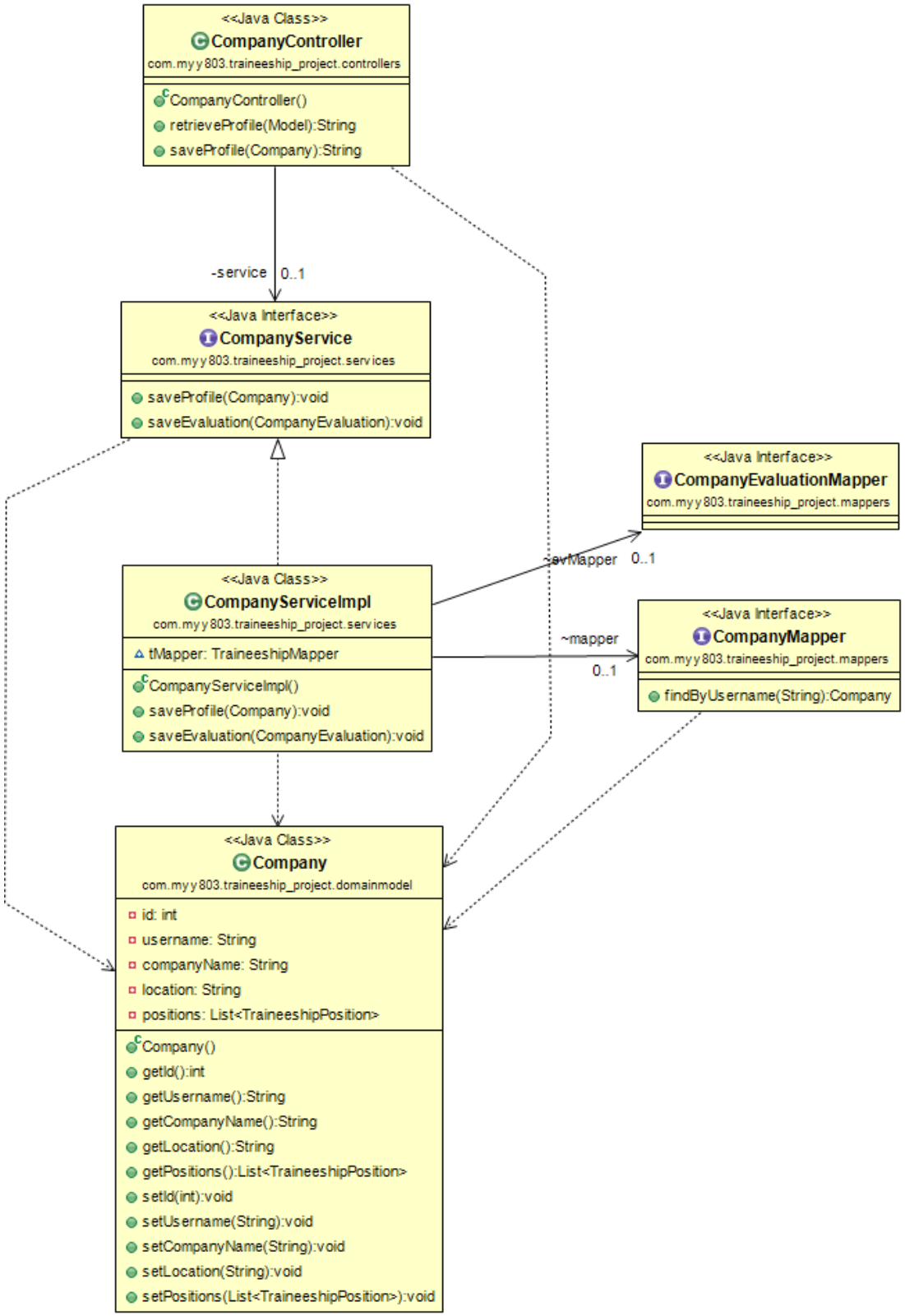
4 Design

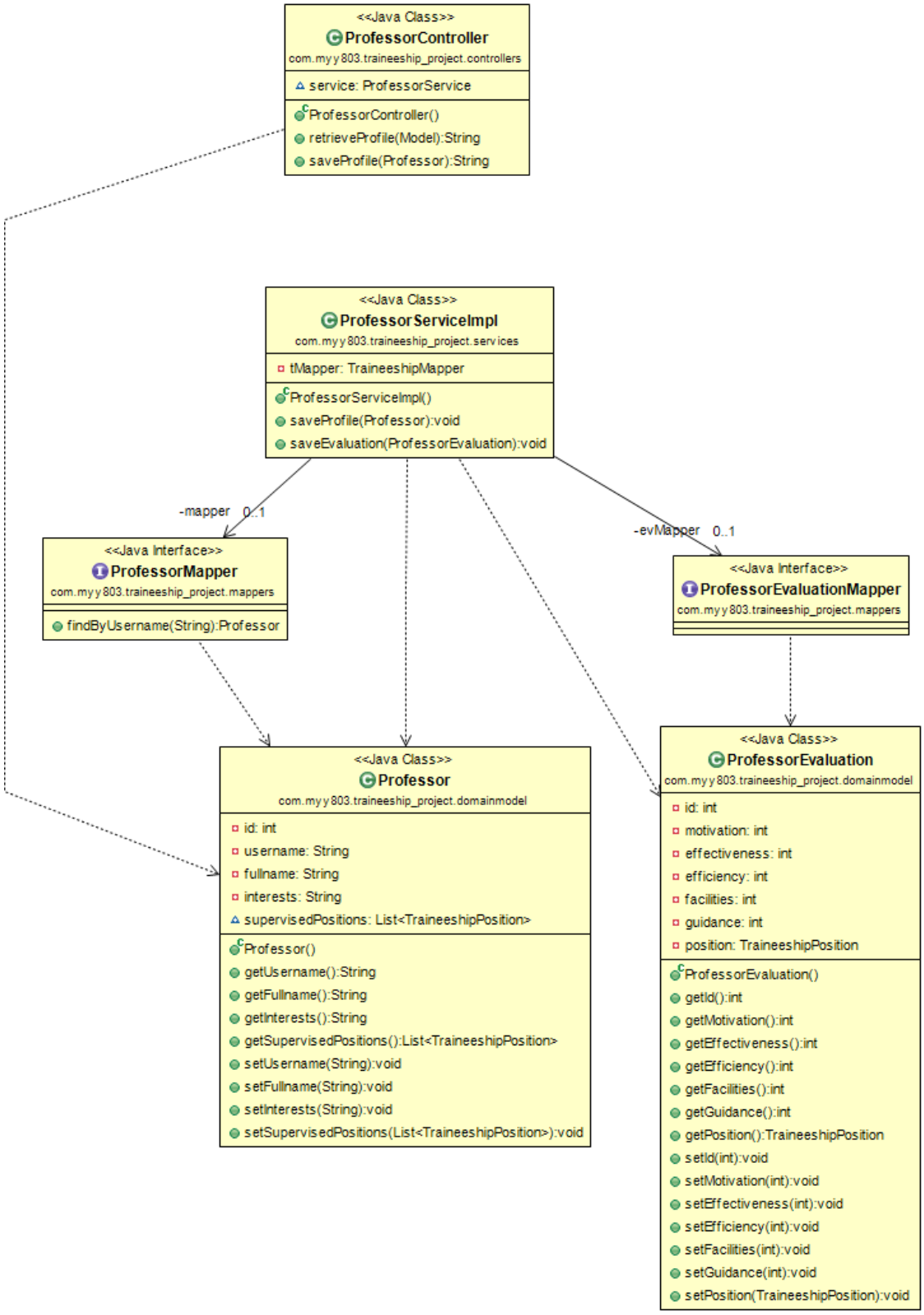
4.1 Architecture

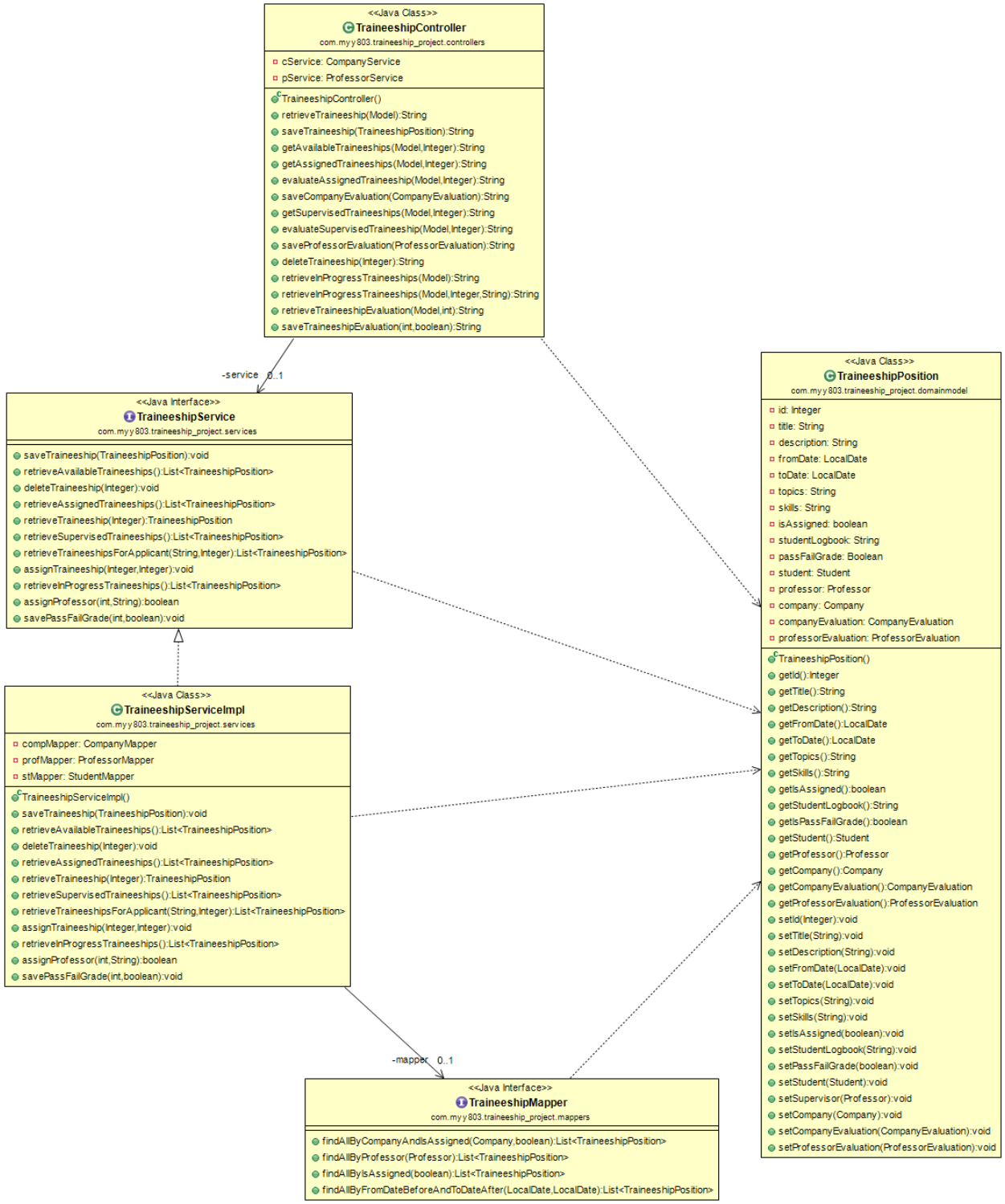


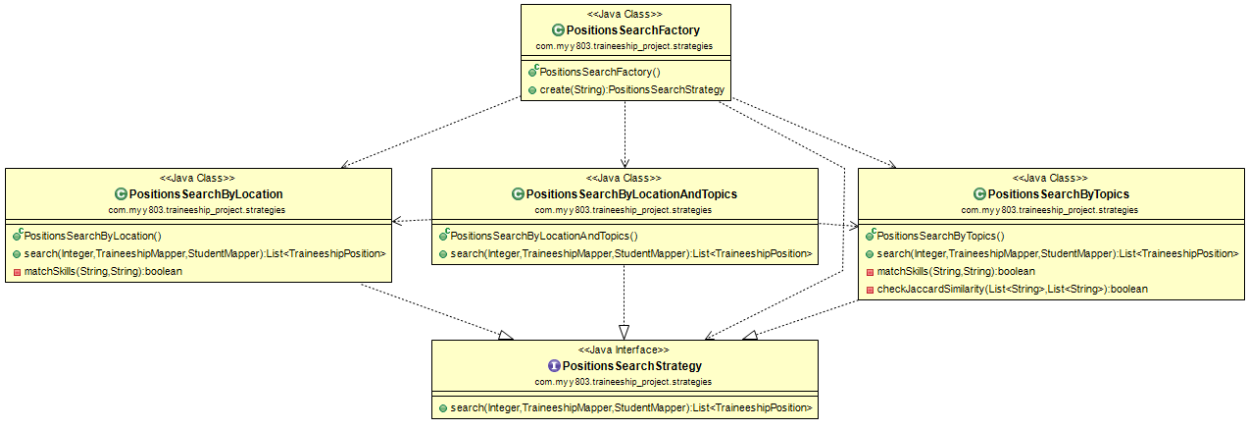
4.2 Design

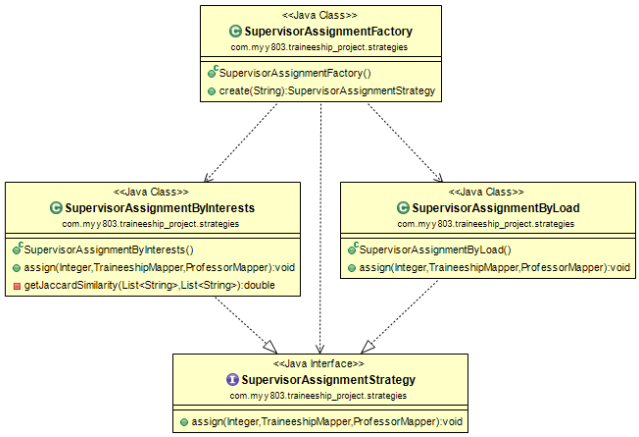

<<Java Class>>
StudentController
com.myy803.traineeship_project.controllers
trainService: TraineeshipService
StudentController()
retrieveProfile(Model):String
saveProfile(Student):String
applyForTraineeship():String
retrieveAppliedUsers(Model,Integer,String,Integer):String
fillLogbook(Model):String
saveLogbook(String):String
-service 0..1
<<Java Interface>>
StudentService
com.myy803.traineeship_project.services
saveProfile(Student):void
applyForTraineeship():void
retrieveAppliedUsers():List<Student>
saveLogbook(String):void
<<Java Interface>>
StudentMapper
com.myy803.traineeship_project.mappers
findByUsername(String):Student
findAllByLookingForTraineeship(boolean):List<Student>
~mapper 0..1
<<Java Class>>
StudentServiceImpl
com.myy803.traineeship_project.services
tMapper: TraineeshipMapper
StudentServiceImpl()
saveProfile(Student):void
applyForTraineeship():void
retrieveAppliedUsers():List<Student>
saveLogbook(String):void
<<Java Class>>
Student
com.myy803.traineeship_project.domainmodel
id: Integer
username: String
fullname: String
AM: String
avgGrade: double
preferredLocation: String
interests: String
skills: String
lookingForTraineeship: boolean
assignedPosition: TraineeshipPosition
Student()
getId():int
getUsername():String
getFullname():String
getAM():String
getAvgGrade():double
getPreferredLocation():String
getInterests():String
getSkills():String
getLookingForTraineeship():boolean
getAssignedPosition():TraineeshipPosition
setId(int):void
setUsername(String):void
setFullname(String):void
setAM(String):void
setAvgGrade(double):void
setPreferredLocation(String):void
setInterests(String):void
setSkills(String):void
setLookingForTraineeship(boolean):void
setAssignedPosition(TraineeshipPosition):void


<<Java Class>>
CompanyController
com.myy803.traineeship_project.controllers
CompanyController()
retrieveProfile(Model):String
saveProfile(Company):String
-service
0..1
<<Java Interface>>
CompanyService
com.myy803.traineeship_project.services
saveProfile(Company):void
saveEvaluation(CompanyEvaluation):void
<<Java Interface>>
CompanyEvaluationMapper
com.myy803.traineeship_project.mappers
~evMapper
0..1
<<Java Class>>
CompanyServiceImpl
com.myy803.traineeship_project.services
tMapper: TraineeshipMapper
CompanyServiceImpl()
saveProfile(Company):void
saveEvaluation(CompanyEvaluation):void
~mapper
0..1
<<Java Interface>>
CompanyMapper
com.myy803.traineeship_project.mappers
findByUsername(String):Company
<<Java Class>>
Company
com.myy803.traineeship_project.domainmodel
id: int
username: String
companyName: String
location: String
positions: List<TraineeshipPosition>
Company()
getId():int
getUsername():String
getCompanyName():String
getLocation():String
getPositions():List<TraineeshipPosition>
setId(int):void
setUsername(String):void
setCompanyName(String):void
setLocation(String):void
setPositions(List<TraineeshipPosition>):void


<<Java Class>>
ProfessorController
com.myy803.traineeship_project.controllers
service: ProfessorService
ProfessorController()
retrieveProfile(Model):String
saveProfile(Professor):String
<<Java Class>>
ProfessorServiceImpl
com.myy803.traineeship_project.services
tMapper: TraineeshipMapper
ProfessorServiceImpl()
saveProfile(Professor):void
saveEvaluation(ProfessorEvaluation):void
-mapper 0..1
-evMapper 0..1
<<Java Interface>>
ProfessorMapper
com.myy803.traineeship_project.mappers
findByUsername(String):Professor
<<Java Interface>>
ProfessorEvaluationMapper
com.myy803.traineeship_project.mappers
<<Java Class>>
Professor
com.myy803.traineeship_project.domainmodel
id: int
username: String
fullname: String
interests: String
supervisedPositions: List<TraineeshipPosition>
Professor()
getUsername():String
getFullname():String
getInterests():String
getSupervisedPositions():List<TraineeshipPosition>
setUsername(String):void
setFullname(String):void
setInterests(String):void
setSupervisedPositions(List<TraineeshipPosition>):void
<<Java Class>>
ProfessorEvaluation
com.myy803.traineeship_project.domainmodel
id: int
motivation: int
effectiveness: int
efficiency: int
facilities: int
guidance: int
position: TraineeshipPosition
ProfessorEvaluation()
getId():int
getMotivation():int
getEffectiveness():int
getEfficiency():int
getFacilities():int
getGuidance():int
getPosition():TraineeshipPosition
setId(int):void
setMotivation(int):void
setEffectiveness(int):void
setEfficiency(int):void
setFacilities(int):void
setGuidance(int):void
setPosition(TraineeshipPosition):void


<<Java Class>>
TraineeshipController
com.my y803.traineeship_project.controllers
cService: CompanyService
pService: ProfessorService
TraineeshipController()
retrieveTraineeship(Model):String
saveTraineeship(TraineeshipPosition):String
getAvailableTraineeships(Model,Integer):String
getAssignedTraineeships(Model,Integer):String
evaluateAssignedTraineeship(Model,Integer):String
saveCompanyEvaluation(CompanyEvaluation):String
getSupervisedTraineeships(Model,Integer):String
evaluateSupervisedTraineeship(Model,Integer):String
saveProfessorEvaluation(ProfessorEvaluation):String
deleteTraineeship(Integer):String
retrieveInProgressTraineeships(Model):String
retrieveInProgressTraineeships(Model,Integer,String):String
retrieveTraineeshipEvaluation(Model,int):String
saveTraineeshipEvaluation(int,boolean):String
-service 0..1
<<Java Interface>>
TraineeshipService
com.my y803.traineeship_project.services
saveTraineeship(TraineeshipPosition):void
retrieveAvailableTraineeships():List<TraineeshipPosition>
deleteTraineeship(Integer):void
retrieveAssignedTraineeships():List<TraineeshipPosition>
retrieveTraineeship(Integer):TraineeshipPosition
retrieveSupervisedTraineeships():List<TraineeshipPosition>
retrieveTraineeshipsForApplicant(String,Integer):List<TraineeshipPosition>
assignTraineeship(Integer,Integer):void
retrieveInProgressTraineeships():List<TraineeshipPosition>
assignProfessor(int,String):boolean
savePassFailGrade(int,boolean):void
<<Java Class>>
TraineeshipServiceImpl
com.my y803.traineeship_project.services
compMapper: CompanyMapper
profMapper: ProfessorMapper
stMapper: StudentMapper
TraineeshipServiceImpl()
saveTraineeship(TraineeshipPosition):void
retrieveAvailableTraineeships():List<TraineeshipPosition>
deleteTraineeship(Integer):void
retrieveAssignedTraineeships():List<TraineeshipPosition>
retrieveTraineeship(Integer):TraineeshipPosition
retrieveSupervisedTraineeships():List<TraineeshipPosition>
retrieveTraineeshipsForApplicant(String,Integer):List<TraineeshipPosition>
assignTraineeship(Integer,Integer):void
retrieveInProgressTraineeships():List<TraineeshipPosition>
assignProfessor(int,String):boolean
savePassFailGrade(int,boolean):void
-mapper 0..1
<<Java Interface>>
TraineeshipMapper
com.my y803.traineeship_project.mappers
findAllByCompanyAndIsAssigned(Company,boolean):List<TraineeshipPosition>
findAllByProfessor(Professor):List<TraineeshipPosition>
findAllByIsAssigned(boolean):List<TraineeshipPosition>
findAllByFromDateBeforeAndToDateAfter(LocalDate,LocalDate):List<TraineeshipPosition>
<<Java Class>>
TraineeshipPosition
com.my y803.traineeship_project.domainmodel
id: Integer
title: String
description: String
fromDate: LocalDate
toDate: LocalDate
topics: String
skills: String
isAssigned: boolean
studentLogbook: String
passFailGrade: Boolean
student: Student
professor: Professor
company: Company
companyEvaluation: CompanyEvaluation
professorEvaluation: ProfessorEvaluation
TraineeshipPosition()
getId():Integer
getTitle():String
getDescription():String
getFromDate():LocalDate
getToDate():LocalDate
getTopics():String
getSkills():String
getIsAssigned():boolean
getStudentLogbook():String
getIsPassFailGrade():boolean
getStudent():Student
getProfessor():Professor
getCompany():Company
getCompanyEvaluation():CompanyEvaluation
getProfessorEvaluation():ProfessorEvaluation
setId(Integer):void
setTitle(String):void
setDescription(String):void
setFromDate(LocalDate):void
setToDate(LocalDate):void
setTopics(String):void
setSkills(String):void
setIsAssigned(boolean):void
setStudentLogbook(String):void
setPassFailGrade(boolean):void
setStudent(Student):void
setSupervisor(Professor):void
setCompany(Company):void
setCompanyEvaluation(CompanyEvaluation):void
setProfessorEvaluation(ProfessorEvaluation):void


<<Java Class>>
PositionsSearchFactory
com.myy803.traineeship_project.strategies
PositionsSearchFactory()
create(String):PositionsSearchStrategy
<<Java Class>>
PositionsSearchByLocation
com.myy803.traineeship_project.strategies
PositionsSearchByLocation()
search(Integer,TraineeshipMapper,StudentMapper):List<TraineeshipPosition>
matchSkills(String,String):boolean
<<Java Class>>
PositionsSearchByLocationAndTopics
com.myy803.traineeship_project.strategies
PositionsSearchByLocationAndTopics()
search(Integer,TraineeshipMapper,StudentMapper):List<TraineeshipPosition>
<<Java Class>>
PositionsSearchByTopics
com.myy803.traineeship_project.strategies
PositionsSearchByTopics()
search(Integer,TraineeshipMapper,StudentMapper):List<TraineeshipPosition>
matchSkills(String,String):boolean
checkJaccardSimilarity(List<String>,List<String>):boolean
<<Java Interface>>
PositionsSearchStrategy
com.myy803.traineeship_project.strategies
search(Integer,TraineeshipMapper,StudentMapper):List<TraineeshipPosition>


<<Java Class>>
SupervisorAssignmentFactory
com.myy803.traineeship_project.strategies
SupervisorAssignmentFactory()
create(String):SupervisorAssignmentStrategy
<<Java Class>>
SupervisorAssignmentByInterests
com.myy803.traineeship_project.strategies
SupervisorAssignmentByInterests()
assign(Integer,TraineeshipMapper,ProfessorMapper):void
getJaccardSimilarity(List<String>,List<String>):double
<<Java Class>>
SupervisorAssignmentByLoad
com.myy803.traineeship_project.strategies
SupervisorAssignmentByLoad()
assign(Integer,TraineeshipMapper,ProfessorMapper):void
<<Java Interface>>
SupervisorAssignmentStrategy
com.myy803.traineeship_project.strategies
assign(Integer,TraineeshipMapper,ProfessorMapper):void

Class Name: TraineeshipProjectApplication	
Responsibilities: ▪ Main class	**Collaborations:** ▪

Class Name: CustomSecuritySuccessHandler	
Responsibilities: ▪ Handles the login request	**Collaborations:** ▪ GrantedAuthority ▪ HttpServletRequest ▪ HttpServletResponse ▪ Authentication

Class Name: WebSecurityConfig	
Responsibilities: ▪ Handles the basic authorization	**Collaborations:** ▪ UserDetailsService ▪ BCryptPasswordEncoder ▪ DaoAuthenticationProvider ▪ HttpSecurity

Class Name: WebMvcConfig	
Responsibilities: ▪ Sets the main view	**Collaborations:** ▪ WebMvcConfigurer

Class Name: CompanyController	
Responsibilities: ▪ Maps the requests that correspond to the company's actions	**Collaborations:** ▪ CompanyService ▪ Company

Class Name: ProfessorController	
Responsibilities: ▪ Maps the requests that correspond to	**Collaborations:** ▪ ProfessorService

the professor's actions	▪ Professor

Class Name: StudentController	
Responsibilities: ▪ Maps the requests that correspond to the student's actions	**Collaborations:** ▪ StudentService ▪ TraineeshipService ▪ Student

Class Name: TraineeshipController	
Responsibilities: ▪ Maps the requests that correspond to the traineeship's actions	**Collaborations:** ▪ TraineeshipService ▪ CompanyService ▪ ProfessorService ▪ TraineeshipPosition ▪ CompanyEvaluation ▪ ProfessorEvaluation

Class Name: UserController	
Responsibilities: ▪ Maps the requests that correspond to the user's actions	**Collaborations:** ▪ UserService ▪ User

Class Name: Company	
Responsibilities: ▪ Domain class for company	**Collaborations:** ▪ TraineeshipPosition

Class Name: CompanyEvaluation

Responsibilities:	Collaborations:
▪ Domain class for company's evaluation	▪ TraineeshipPosition

Class Name: Interest	
Responsibilities:	**Collaborations:**
▪ Enum for interests	▪ -

Class Name: Professor	
Responsibilities:	**Collaborations:**
▪ Domain class for professor	▪ TraineeshipPosition

Class Name: ProfessorEvaluation	
Responsibilities:	**Collaborations:**
▪ Domain class for professor's evaluation	▪ TraineeshipPosition

Class Name: Role	
Responsibilities:	**Collaborations:**
▪ Enum for user roles	▪ -

Class Name: Skill	
Responsibilities:	**Collaborations:**
▪ Enum for skills	▪ -

Class Name: Student	
Responsibilities:	**Collaborations:**
▪ Domain class for student	▪ TraineeshipPosition

Class Name: User	
Responsibilities:	**Collaborations:**

▪ Domain class for user	▪ UserDetails ▪ Role

Class Name: TraineeshipPosition	
Responsibilities: ▪ Domain class for traineeship position	**Collaborations:** ▪ Student ▪ Professor ▪ Company ▪ CompanyEvaluation ▪ ProfessorEvaluation

Class Name: CompanyEvaluationMapper	
Responsibilities: ▪ Data access for CompanyEvaluation objects	**Collaborations:** ▪ JpaRepository ▪ CompanyEvaluation

Class Name: CompanyMapper	
Responsibilities: ▪ Data access for Company objects	**Collaborations:** ▪ JpaRepository ▪ Company

Class Name: ProfessorEvaluationMapper	
Responsibilities: ▪ Data access for ProfessorEvaluation objects	**Collaborations:** ▪ JpaRepository ▪ ProfessorEvaluation

Class Name: ProfessorMapper	
Responsibilities: ▪ Data access for Professor objects	**Collaborations:** ▪ JpaRepository

	▪ Professor

Class Name: StudentMapper	
Responsibilities: ▪ Data access for Student objects	**Collaborations:** ▪ JpaRepository ▪ Student

Class Name: TraineeshipMapper	
Responsibilities: ▪ Data access for TraineeshipPosition objects	**Collaborations:** ▪ JpaRepository ▪ TraineeshipPosition ▪ Company ▪ Professor

Class Name: UserMapper	
Responsibilities: ▪ Data access for User objects	**Collaborations:** ▪ JpaRepository ▪ User

Class Name: CompanyService	
Responsibilities: ▪ Interface that contains the company actions	**Collaborations:** ▪ Company ▪ CompanyEvaluation

Class Name: CompanyServiceImpl	
Responsibilities: ▪ Implementation of the CompanyService interface	**Collaborations:** ▪ CompanyService ▪ CompanyMapper ▪ CompanyEvaluationMapper ▪ TraineeshipMapper

	▪ Company ▪ CompanyEvaluation

Class Name: ProfessorService	
Responsibilities: ▪ Interface that contains the professor actions	**Collaborations:** ▪ Professor ▪ ProfessorEvaluation

Class Name: ProfessorServiceImpl	
Responsibilities: ▪ Implementation of the ProfessorService interface	**Collaborations:** ▪ ProfessorService ▪ ProfessorMapper ▪ ProfessorEvaluationMapper ▪ TraineeshipMapper ▪ Professor ▪ ProfessorEvaluation

Class Name: StudentService	
Responsibilities: ▪ Interface that contains the student actions	**Collaborations:** ▪ Student

Class Name: StudentServiceImpl	
Responsibilities: ▪ Implementation of the StudentService interface	**Collaborations:** ▪ StudentService ▪ StudentMapper ▪ TraineeshipMapper

	▪ Student

Class Name: TraineeshipService	
Responsibilities: ▪ Interface that contains the traineeship position actions	**Collaborations:** ▪ TraineeshipPosition

Class Name: TraineeshipServiceImpl	
Responsibilities: ▪ Implementation of the TraineeshipService interface	**Collaborations:** ▪ TraineeshipMapper ▪ CompanyMapper ▪ ProfessorMapper ▪ StudentMapper ▪ TraineeshipPosition ▪ PositionsSearchFactory ▪ PositionsSearchStrategy ▪ SupervisorAssignmentFactory ▪ SupervisorAssignmentStrategy

Class Name: UserService	
Responsibilities: ▪ Interface that contains the user actions	**Collaborations:** ▪ User

Class Name: UserServiceImpl	
Responsibilities: ▪ Implementation of the UserService interface	**Collaborations:** ▪ UserService ▪ UserDetailsService ▪ BCryptPasswordEncoder ▪ UserMapper ▪ User

Class Name: PositionsSearchByLocation	
Responsibilities: ▪ Implementation of the PositionsSearchStrategy interface that returns results based on location	**Collaborations:** ▪ PositionsSearchStrategy ▪ TraineeshipPosition ▪ TraineeshipMapper ▪ StudentMapper ▪

Class Name: PositionsSearchByLocationAndTopics	
Responsibilities: ▪ Implementation of the PositionsSearchStrategy interface that returns results based on location and topics	**Collaborations:** ▪ PositionsSearchStrategy ▪ TraineeshipPosition ▪ TraineeshipMapper ▪ StudentMapper ▪ PositionsSearchByTopics ▪ PositionsSearchByLocation

Class Name: PositionsSearchByTopics	
Responsibilities: ▪ Implementation of the PositionsSearchStrategy interface that returns results based on topics	**Collaborations:** ▪ PositionsSearchStrategy ▪ TraineeshipPosition ▪ TraineeshipMapper ▪ StudentMapper ▪ Student

Class Name: PositionsSearchFactory	
Responsibilities: ▪ Factory for the PositionsSearchStrategy objects	**Collaborations:** ▪ PositionsSearchStrategy ▪ PositionsSearchByLocation ▪ PositionsSearchByTopics

	▪ PositionsSearchByLocationAndTopics

Class Name: PositionsSearchStrategy	
Responsibilities: ▪ Interface for the search actions	**Collaborations:** ▪ TraineeshipPosition ▪ TraineeshipMapper ▪ StudentMapper

Class Name: SupervisorAssignmentByInterests	
Responsibilities: ▪ Implementation of the SupervisorAssignmentStrategy interface that assigns professor based on interests	**Collaborations:** ▪ SupervisorAssignmentStrategy ▪ TraineeshipMapper ▪ ProfessorMapper ▪ Professor ▪ TraineeshipPosition

Class Name: SupervisorAssignmentByLoad	
Responsibilities: ▪ Implementation of the SupervisorAssignmentStrategy interface that assigns professor based on load	**Collaborations:** ▪ SupervisorAssignmentStrategy ▪ TraineeshipMapper ▪ ProfessorMapper ▪ Professor ▪ TraineeshipPosition

Class Name: SupervisorAssignmentFactory	
Responsibilities: ▪ Factory for the SupervisorAssignmentStrategy objects	**Collaborations:** ▪ SupervisorAssignmentStrategy ▪ SupervisorAssignmentByInterests ▪ SupervisorAssignmentByLoad

Class Name: SupervisorAssignmentStrategy	
Responsibilities: ▪ Interface for the supervisor assignment	**Collaborations:** ▪ TraineeshipMapper ▪ ProfessorMapper